# Functional Specification: AXI4 DMA & AXI4 RAM Subsystem

> **Attribution:** The base RTL for this subsystem is derived from Alex Forencich's popular open-source `verilog-axi` GitHub repository.

**Document Version:** 1.0
**Target RTL Modules:**
- `axi_dma.v` (Top-level DMA Engine) - `axi_dma_rd.v` (Memory-Mapped to Stream Read Engine) - `axi_dma_wr.v` (Stream to Memory-Mapped Write Engine) - `axi_ram.v` (AXI4 Memory Slave Block)

---

## 1. System Overview & Architecture

The AXI4 DMA subsystem provides high-throughput, bidirectional Direct Memory Access between an **AXI4 Memory-Mapped (MM)** interconnect/slave (such as `axi_ram`) and an **AXI4-Stream (AXIS)** interface.

The design consists of: 1. **`axi_dma_rd` (MM2S Channel):** Fetches data from AXI4 memory via Read Address (`AR`) & Read Data (`R`) channels and transmits it as an AXI-Stream. 2. **`axi_dma_wr` (S2MM Channel):** Receives data from an AXI-Stream and writes it to AXI4 memory via Write Address (`AW`), Write Data (`W`), and Write Response (`B`) channels. 3. **`axi_ram` (AXI4 Slave Memory):** A parameterized single-port/dual-channel memory model supporting full AXI4 bursts (`INCR`, `WRAP`, `FIXED`), byte-lane enables, and pipelined output registers.

### Block Diagram

```
                 ┌────────────────────────────────────────────────────────┐
                 │                        AXI4 DMA                        │
                 │                                                        │
[Read Desc] ───> │  ┌──────────────┐                  ┌──────────────────┐│
                 │  │              │ ── AR Channel ─> │                  ││
[Read Stream]<── │  │ axi_dma_rd   │ <── R Channel ── │                  ││
                 │  │ (MM2S Engine)│                  │                  ││
[Read Status]<── │  └──────────────┘                  │                  ││
                 │                                    │     AXI4 Bus     ││  ====> [ axi_ram ]
[Write Desc]───> │  ┌──────────────┐                  │   Interconnect   ││  <==== (AXI4 Slave Memory)
                 │  │              │ ── AW Channel ─> │                  ││
[Write Stream]─> │  │ axi_dma_wr   │ ── W Channel ──> │                  ││
                 │  │ (S2MM Engine)│ <── B Channel ── │                  ││
[Write Status]<─ │  └──────────────┘                  └──────────────────┘│
                 └────────────────────────────────────────────────────────┘
```

---

## 2. Parameter Definitions & Configuration Options

| Parameter | Default | Valid Range | Description |
|---|---|---|---|
| `AXI_DATA_WIDTH` | `32` | `32, 64, 128, 256, 512, 1024` | Width of the AXI4 data bus in bits. Must be an even power of 2. |
| `AXI_ADDR_WIDTH` | `16` | `12` to `64` | Width of the AXI4 address bus in bits. |
| `AXI_STRB_WIDTH` | `DATA_WIDTH/8` | Computed | Number of byte strobe signals (1 bit per byte). |
| `AXI_ID_WIDTH` | `8` | `1` to `32` | Width of AXI transaction ID signals (`AWID`, `BID`, `ARID`, `RID`). |

| | | | |
|---|---|---|---|
| `AXI_MAX_BURST_LEN` | `16` | `1` to `256` | Maximum burst length (in beats) generated by the DMA per AXI transfer. |
| `AXIS_DATA_WIDTH` | `AXI_DATA_WIDTH` | Equal to `AXI_DATA_WIDTH` | Width of AXI-Stream data payloads. |
| `AXIS_KEEP_ENABLE` | `(DATA_WIDTH>8)` | `0, 1` | Enables byte qualification (`TKEEP`) on AXI-Stream interfaces. |
| `AXIS_KEEP_WIDTH` | `DATA_WIDTH/8` | Computed | Width of `TKEEP` signal in bits. |
| `AXIS_LAST_ENABLE` | `1` | `0, 1` | Enables frame termination (`TLAST`) generation on stream interfaces. |
| `AXIS_ID_ENABLE` | `0` | `0, 1` | Enables propagation of stream `TID`. |
| `AXIS_ID_WIDTH` | `8` | `1` to `32` | Width of stream `TID`. |
| `AXIS_DEST_ENABLE` | `0` | `0, 1` | Enables propagation of stream routing `TDEST`. |
| `AXIS_DEST_WIDTH` | `8` | `1` to `32` | Width of stream `TDEST`. |
| `AXIS_USER_ENABLE` | `1` | `0, 1` | Enables propagation of user sideband `TUSER`. |
| `AXIS_USER_WIDTH` | `1` | `1` to `32` | Width of stream `TUSER`. |
| `LEN_WIDTH` | `20` | `8` to `32` | Width of descriptor byte-transfer length field (Supports transfers up to 2^20-1 bytes). |
| `TAG_WIDTH` | `8` | `1` to `16` | Width of descriptor transaction tracking tag. |
| `ENABLE_SG` | `0` | `0, 1` | **Scatter-Gather:** Allows multiple descriptors to compose a single AXI stream frame (suppresses intermediate `TLAST`). |
| `ENABLE_UNALIGNED` | `0` | `0, 1` | **Unaligned Transfers:** Allows starting addresses not aligned to `AXI_DATA_WIDTH/8` byte boundary. |
| `PIPELINE_OUTPUT` | `0` | `0, 1` | *(axi_ram only)* Inserts an additional output register stage on read data path. |

# 3. Port List & Signal Descriptions

## 3.1 Global Clock and Reset

| **Signal Name** | **Dir** | **Width** | **Description** |
|---|---|---|---|
| `clk` | Input | 1 | Global system clock. All interfaces operate synchronously on the rising edge. |
| `rst` | Input | 1 | Synchronous active-high reset. Forces all internal FSMs to `IDLE` and resets handshakes to `0`. |

## 3.2 Read Descriptor Interface (Host → DMA)

Used by host/CPU to command the DMA Read Channel to fetch data from AXI memory into the AXI-Stream.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `s_axis_read_desc_addr` | Input | `AXI_ADDR_WIDTH` | Starting byte address in AXI memory space. |
| `s_axis_read_desc_len` | Input | `LEN_WIDTH` | Total number of bytes to transfer. |
| `s_axis_read_desc_tag` | Input | `TAG_WIDTH` | Identifier tag returned in status output upon completion. |
| `s_axis_read_desc_id` | Input | `AXIS_ID_WIDTH` | `TID` value to attach to outgoing stream packets. |
| `s_axis_read_desc_dest` | Input | `AXIS_DEST_WIDTH` | `TDEST` value to attach to outgoing stream packets. |
| `s_axis_read_desc_user` | Input | `AXIS_USER_WIDTH` | `TUSER` value to attach to outgoing stream packets. |
| `s_axis_read_desc_valid` | Input | 1 | Indicates a valid read descriptor is present. |
| `s_axis_read_desc_ready` | Output | 1 | Indicates the DMA read engine can accept the descriptor. |

---

## 3.3 Read Descriptor Status Interface (DMA → Host)

Signals completion or failure of a read transfer.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axis_read_desc_status_tag` | Output | `TAG_WIDTH` | Tag matching the completed read descriptor. |
| `m_axis_read_desc_status_error` | Output | 4 | Status/error code for the completed transfer (See Section 6). |
| `m_axis_read_desc_status_valid` | Output | 1 | Pulsed high for 1 cycle when status is valid. |

---

## 3.4 AXI-Stream Read Data Interface (DMA → Sink)

Streams out data read from AXI memory.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axis_read_data_tdata` | Output | `AXIS_DATA_WIDTH` | Read data payload. |
| `m_axis_read_data_tkeep` | Output | `AXIS_KEEP_WIDTH` | Byte qualifiers indicating valid byte lanes. |
| `m_axis_read_data_tvalid` | Output | 1 | Indicates valid stream data is presented. |
| `m_axis_read_data_tready` | Input | 1 | Downstream sink ready to consume data. |
| `m_axis_read_data_tlast` | Output | 1 | Asserted on the last beat of a packet frame. |
| `m_axis_read_data_tid` | Output | `AXIS_ID_WIDTH` | Stream ID copied from descriptor. |
| `m_axis_read_data_tdest` | Output | `AXIS_DEST_WIDTH` | Stream Destination copied from descriptor. |
| `m_axis_read_data_tuser` | Output | `AXIS_USER_WIDTH` | Stream User sideband copied from descriptor. |

## 3.5 Write Descriptor Interface (Host → DMA)

Used by host/CPU to command the DMA Write Channel to receive stream data and write it to AXI memory.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `s_axis_write_desc_addr` | Input | `AXI_ADDR_WIDTH` | Starting byte address in AXI memory space. |
| `s_axis_write_desc_len` | Input | `LEN_WIDTH` | Maximum expected transfer length in bytes. |
| `s_axis_write_desc_tag` | Input | `TAG_WIDTH` | Identifier tag returned in status output upon completion. |
| `s_axis_write_desc_valid` | Input | 1 | Indicates a valid write descriptor is present. |
| `s_axis_write_desc_ready` | Output | 1 | Indicates DMA write engine can accept the descriptor. |

## 3.6 Write Descriptor Status Interface (DMA → Host)

Reports write transfer completion, actual byte count transferred, and captured stream sidebands.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axis_write_desc_status_len` | Output | `LEN_WIDTH` | Actual number of bytes written to memory. |
| `m_axis_write_desc_status_tag` | Output | `TAG_WIDTH` | Tag matching the completed write descriptor. |
| `m_axis_write_desc_status_id` | Output | `AXIS_ID_WIDTH` | `TID` captured from incoming stream packet. |
| `m_axis_write_desc_status_dest` | Output | `AXIS_DEST_WIDTH` | `TDEST` captured from incoming stream packet. |
| `m_axis_write_desc_status_user` | Output | `AXIS_USER_WIDTH` | `TUSER` captured from incoming stream packet. |
| `m_axis_write_desc_status_error` | Output | 4 | Status/error code (See Section 6). |
| `m_axis_write_desc_status_valid` | Output | 1 | Pulsed high for 1 cycle when status is valid. |

## 3.7 AXI-Stream Write Data Interface (Source → DMA)

Receives incoming stream data to be written into AXI memory.

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `s_axis_write_data_tdata` | Input | `AXIS_DATA_WIDTH` | Incoming data payload. |
| `s_axis_write_data_tkeep` | Input | `AXIS_KEEP_WIDTH` | Incoming byte valid qualifiers. |
| `s_axis_write_data_tvalid` | Input | 1 | Stream data valid. |
| `s_axis_write_data_tready` | Output | 1 | DMA ready to accept stream data. |
| `s_axis_write_data_tlast` | Input | 1 | Asserted by source on the final beat of a frame. |

| | | | |
|---|---|---|---|
| `s_axis_write_data_tid` | Input | `AXIS_ID_WIDTH` | Incoming Stream ID. |
| `s_axis_write_data_tdest` | Input | `AXIS_DEST_WIDTH` | Incoming Stream Destination. |
| `s_axis_write_data_tuser` | Input | `AXIS_USER_WIDTH` | Incoming Stream User sideband. |

## 3.8 AXI4 Master Interface (DMA → AXI Interconnect / RAM)

### Write Address Channel (AW)

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axi_awid` | Output | `AXI_ID_WIDTH` | Write transaction ID. |
| `m_axi_awaddr` | Output | `AXI_ADDR_WIDTH` | Write start address. |
| `m_axi_awlen` | Output | 8 | Burst length: `0` to `AXI_MAX_BURST_LEN-1` (denoting 1 to `AXI_MAX_BURST_LEN` beats). |
| `m_axi_awsize` | Output | 3 | Burst size: log2(AXI_DATA_WIDTH/8) bytes per beat. |
| `m_axi_awburst` | Output | 2 | Burst type: `2'b01` (`INCR`). |
| `m_axi_awvalid` | Output | 1 | Write address valid. |
| `m_axi_awready` | Input | 1 | Slave ready to accept write address. |

### Write Data Channel (W)

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axi_wdata` | Output | `AXI_DATA_WIDTH` | Write data bus. |
| `m_axi_wstrb` | Output | `AXI_STRB_WIDTH` | Write byte strobes. |
| `m_axi_wlast` | Output | 1 | Asserted on final beat of write burst. |
| `m_axi_wvalid` | Output | 1 | Write data valid. |
| `m_axi_wready` | Input | 1 | Slave ready to accept write data. |

### Write Response Channel (B)

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axi_bid` | Input | `AXI_ID_WIDTH` | Response transaction ID. |
| `m_axi_bresp` | Input | 2 | Response status (`2'b00` OKAY, `2'b10` SLVERR, `2'b11` DECERR). |
| `m_axi_bvalid` | Input | 1 | Write response valid. |
| `m_axi_bready` | Output | 1 | DMA master ready to accept write response. |

### Read Address Channel (AR)

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| `m_axi_arid` | Output | `AXI_ID_WIDTH` | Read transaction ID. |
| `m_axi_araddr` | Output | `AXI_ADDR_WIDTH` | Read start address. |
| `m_axi_arlen` | Output | 8 | Burst length: `0` to `AXI_MAX_BURST_LEN-1`. |
| `m_axi_arsize` | Output | 3 | Burst size: log2(AXI_DATA_WIDTH/8) bytes per beat. |
| `m_axi_arburst` | Output | 2 | Burst type: `2'b01` (`INCR`). |
| `m_axi_arvalid` | Output | 1 | Read address valid. |
| `m_axi_arready` | Input | 1 | Slave ready to accept read address. |

**Read Data Channel (R)**

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| m_axi_rid | Input | AXI_ID_WIDTH | Read data ID tag. |
| m_axi_rdata | Input | AXI_DATA_WIDTH | Read data bus. |
| m_axi_rresp | Input | 2 | Read response status. |
| m_axi_rlast | Input | 1 | Asserted on final beat of read burst. |
| m_axi_rvalid | Input | 1 | Read data valid. |
| m_axi_rready | Output | 1 | DMA master ready to accept read data. |

---

### 3.9 Control & Configuration Ports

| Signal Name | Dir | Width | Description |
|---|---|---|---|
| read_enable | Input | 1 | Master enable for Read DMA engine. When 0, descriptor processing halts. |
| write_enable | Input | 1 | Master enable for Write DMA engine. When 0, descriptor processing halts. |
| write_abort | Input | 1 | Immediately aborts the current write operation and drops incoming stream data. |

---

# 4. Theory of Operation & Detailed Functional Description

## 4.1 Read Channel (MM2S: Memory-to-Stream) Operation

1. **Descriptor Acceptance:** Host asserts `s_axis_read_desc_valid`. If `read_enable` is high, the DMA accepts the descriptor by asserting `s_axis_read_desc_ready`.
2. **Burst Calculation & 4KB Boundary Splitting:**
   - The DMA calculates the number of AXI bursts needed based on `s_axis_read_desc_len` and `AXI_MAX_BURST_LEN`.
   - **Rule:** If a transfer spans across a 4KB address boundary (`addr[11:0] + len > 4096`), the DMA automatically cuts the current burst at `0xFFF` and generates a subsequent burst starting at `0x000` of the next page.
3. **AXI Read Issuance:** DMA issues Read Address requests on `AR` channel (`m_axi_arvalid`).
4. **Data Forwarding & Repacking:**
   - Read data arriving on `m_axi_rdata` is buffered in internal FIFOs.
   - If `ENABLE_UNALIGNED` is set, data is shifted/realigned to the requested byte offset.
   - Transmitted onto `m_axis_read_data_*`.
5. **Frame Completion:**
   - On the final byte of the requested length, `m_axis_read_data_tlast` is asserted (unless `ENABLE_SG=1` and further descriptors are queued).
   - Status is emitted on `m_axis_read_desc_status_*` with `m_axis_read_desc_status_valid = 1`.

---

## 4.2 Write Channel (S2MM: Stream-to-Memory) Operation

1. **Descriptor Acceptance:** Host issues a write descriptor with memory target `addr` and max buffer `len`.
2. **Stream Ingestion:** DMA asserts `s_axis_write_data_tready` and accepts incoming packet data.
3. **AXI Write Issuance:**
   - DMA issues `m_axi_awaddr` / `m_axi_awlen` on `AW` channel.
   - Drives payload onto `m_axi_wdata` with appropriate `m_axi_wstrb`.
   - Asserts `m_axi_wlast` on the last beat of each segmented burst.
4. **Termination Scenarios:**
   - **Normal Finish:** Packet completes when `s_axis_write_data_tlast` is received.
   - **Length Exhausted:** If incoming bytes reach descriptor `len` before `tlast`, DMA finishes the write and transitions to drop remaining stream bytes until `tlast` is seen.
   - **Abort:** If `write_abort` is asserted, DMA halts write generation and drains the stream.

5. **Response Collection:** DMA collects BVALID/BRESP for every issued write burst before generating m_axis_write_desc_status_valid.

---

### 4.3 AXI4 RAM Slave (axi_ram) Operation

1. **Independent Write & Read FSMs:** Supports simultaneous read and write operations.
2. **Burst Types Supported:**
    - 2'b00 (FIXED): Address remains static for all beats (used for FIFO emulation).
    - 2'b01 (INCR): Address increments by 2^AxSIZE on every beat.
    - 2'b10 (WRAP): Address increments and wraps at the cache-line boundary (2^AxSIZE × AxLEN).
3. **Byte Masking:** Applies s_axi_wstrb on write operations so unselected bytes in memory retain their previous contents.
4. **Pipelining:** If PIPELINE_OUTPUT = 1, read output is registered through an extra flip-flop stage for improved Fmax.

---

# 5. Protocol Error Codes Matrix

When a transfer completes, the status error bus (m_axis_*_desc_status_error[3:0]) indicates execution outcome:

| Error Code (3:0) | Symbolic Constant | Description |
|---|---|---|
| 4'd0 | DMA_ERROR_NONE | Transfer completed successfully with OKAY response. |
| 4'd1 | DMA_ERROR_TIMEOUT | Timeout occurred waiting for channel handshake. |
| 4'd2 | DMA_ERROR_PARITY | Parity error detected on payload. |
| 4'd4 | DMA_ERROR_AXI_RD_SLVERR | AXI Slave returned SLVERR (2'b10) on Read data (RRESP). |
| 4'd5 | DMA_ERROR_AXI_RD_DECERR | AXI Interconnect returned DECERR (2'b11) on Read data (RRESP). |
| 4'd6 | DMA_ERROR_AXI_WR_SLVERR | AXI Slave returned SLVERR (2'b10) on Write response (BRESP). |
| 4'd7 | DMA_ERROR_AXI_WR_DECERR | AXI Interconnect returned DECERR (2'b11) on Write response (BRESP). |
| 4'd8 | DMA_ERROR_PCIE_FLR | PCIe Function Level Reset triggered. |

---

# 6. Critical Boundary & Timing Rules

1. **4KB Address Boundary Rule (AXI4 Spec Section A3.4.1):**
    - An AXI burst must **never** cross a 4KB address boundary. The DMA engine must partition any transfer crossing 0xX000 into two or more independent bursts.
2. **Handshake Stability (AXI4 Spec Section A3.2.1):**
    - Once a VALID signal (AWVALID, WVALID, ARVALID, TVALID, status_valid) is asserted, it **must remain asserted** and its associated payload **must remain unchanged** until the corresponding READY signal is sampled high.
3. **Write Response Ordering:**
    - Status valid for a write descriptor must **not** be asserted until all BRESP responses for all generated bursts of that descriptor have been received.